Gibbs example: coal-mining disasters

Coal-mining disaster change-point example

We observe yearly counts

$$y_i = \text{number of coal-mining disasters in year } i,$$

and assume that the Poisson rate changes at an unknown year M:

$$Y_i \sim \text{Poisson}(\lambda_1), \qquad i = 1, \ldots, M,$$

and

$$Y_i \sim \text{Poisson}(\lambda_2), \qquad i = M+1, \ldots, n.$$

The Gibbs sampler updates λ_1, λ_2, and M from their full conditional distributions.

load data

The data set records the times of coal-mining disasters. The code converts these times to calendar years and then counts how many disasters occurred in each year. Thus, the vector `freq` contains the observed values $y_1, \ldots, y_n$.

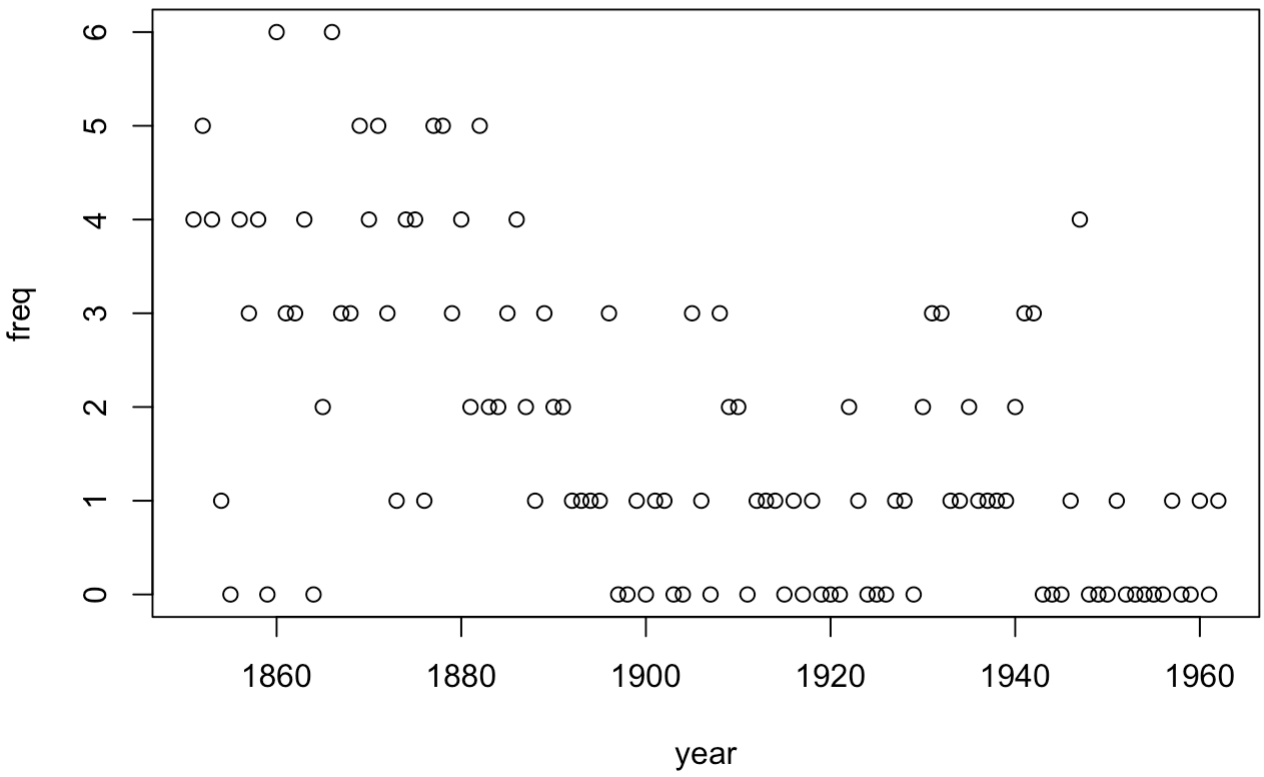
```
library(boot)
data(coal)
str(coal)
```

```
'data.frame':   191 obs. of  1 variable:
 $ date: num  1851 1852 1852 1852 1852 ...
```

```
when <- floor(coal)
year <- 1851:1962
freq <- sapply(year, function(x, y) sum(y==x), y=when)
str(freq)
```

```
 int [1:112] 4 5 4 1 0 4 3 4 0 6 ...
```

```
n <- length(freq)
plot(year, freq)
```



In particular,

$$\texttt{freq[i]} = y_i.$$

Implement Gibbs Sampler

The priors used in the code are

$$\lambda_1 \sim \text{Gamma}(a_1, b_1), \qquad \lambda_2 \sim \text{Gamma}(a_2, b_2),$$

with

$$a_1 = a_2 = 0.5, \qquad b_1 = b_2 = 0.001,$$

and a uniform prior for M.

The full conditional for λ_1 is

$$\lambda_1 \mid \lambda_2, M, \mathbf{y} \sim \text{Gamma}\left(a_1 + \sum_{i=1}^{M} y_i,\ b_1 + M\right).$$

Similarly,

$$\lambda_2 \mid \lambda_1, M, \mathbf{y} \sim \text{Gamma}\left(a_2 + \sum_{i=M+1}^{n} y_i,\ b_2 + n - M\right).$$

For M,

$$p(M = m \mid \lambda_1, \lambda_2, \mathbf{y}) \propto \left(\frac{\lambda_1}{\lambda_2}\right)^{\sum_{i=1}^{m} y_i} \exp\{(\lambda_2 - \lambda_1)m\}.$$

Why `cfreq <- cumsum(freq)`?

The conditional distributions repeatedly require sums such as

$$\sum_{i=1}^{M} y_i.$$

Since `freq` contains the y_i values, `cumsum(freq)` creates all partial sums at once:

$$\texttt{cfreq[m]} = \sum_{i=1}^{m} y_i.$$

Therefore,

$$\sum_{i=1}^{M} y_i = \texttt{cfreq[M]},$$

and the number of disasters after the change point is

$$\sum_{i=M+1}^{n} y_i = \sum_{i=1}^{n} y_i - \sum_{i=1}^{M} y_i = \texttt{cfreq[n] - cfreq[M]}.$$

This is why the two Gamma updates in the R code use `cfreq[M]` and `cfreq[n] - cfreq[M]`.

The code below is unchanged from the original Rmd.

```
GibbsS <- function(la1, la2, M, nreps, freq){

  # priors; uniform for M
  a1 <- .5
  b1 <- .001
  a2 <- .5
  b2 <- .001

  Gsamples <- matrix(nrow=nreps, ncol=3)
  Gsamples[1,] <- c(la1, la2, M)

  # main loop
  cfreq <- cumsum(freq)
  for (i in 2:nreps) {
    la1 <- rgamma(1, a1 + cfreq[M], b1 + M)
    la2 <- rgamma(1, a2 + cfreq[n] - cfreq[M], b2 + n - M)
    M <- sample(1:n, 1, prob=(la1/la2)^cfreq*exp((la2-la1)*(1:n)))
    Gsamples[i,] <- c(la1, la2, M)
  }

  return(Gsamples=Gsamples)
}
```

Connecting each line to the formula

The first Gibbs update is

```
la1 <- rgamma(1, a1 + cfreq[M], b1 + M)
```

because

$$a_1 + \texttt{cfreq[M]} = a_1 + \sum_{i=1}^{M} y_i.$$

The second update is

```
la2 <- rgamma(1, a2 + cfreq[n] - cfreq[M], b2 + n - M)
```

because

$$\texttt{cfreq[n]} - \texttt{cfreq[M]} = \sum_{i=M+1}^{n} y_i.$$

For the change point, the code uses

```
(la1/la2)^cfreq*exp((la2-la1)*(1:n))
```

Here, `cfreq` is the vector

$$\left(\sum_{i=1}^{1} y_i, \sum_{i=1}^{2} y_i, \ldots, \sum_{i=1}^{n} y_i \right),$$

while `1:n` is the vector of all possible values $m = 1, \ldots, n$. R therefore calculates one unnormalised posterior weight for every possible value of M:

$$w_m = \left(\frac{\lambda_1}{\lambda_2} \right)^{\sum_{i=1}^{m} y_i} \exp\{(\lambda_2 - \lambda_1)m\}.$$

Then

```
M <- sample(1:n, 1, prob=...)
```

samples one value of M using these weights. The weights do not need to be normalised beforehand because `sample()` normalises the `prob` argument internally.

Simulate two Markov Chains (of length 500) using Gibbs Sampler

Two chains are started from different initial values. If both chains settle into the same region, this provides a useful visual check that the results are not being driven by the chosen starting point.

```
# sample size
nreps <- 500

# initial values
la1 <- 1
la2 <- 1
M <- 100
```

```
GibbsS1 = GibbsS(la1, la2, M, nreps, freq)

# initial values
la1 <- 3
la2 <- 3
M <- 50

GibbsS2 = GibbsS(la1, la2, M, nreps, freq)
```

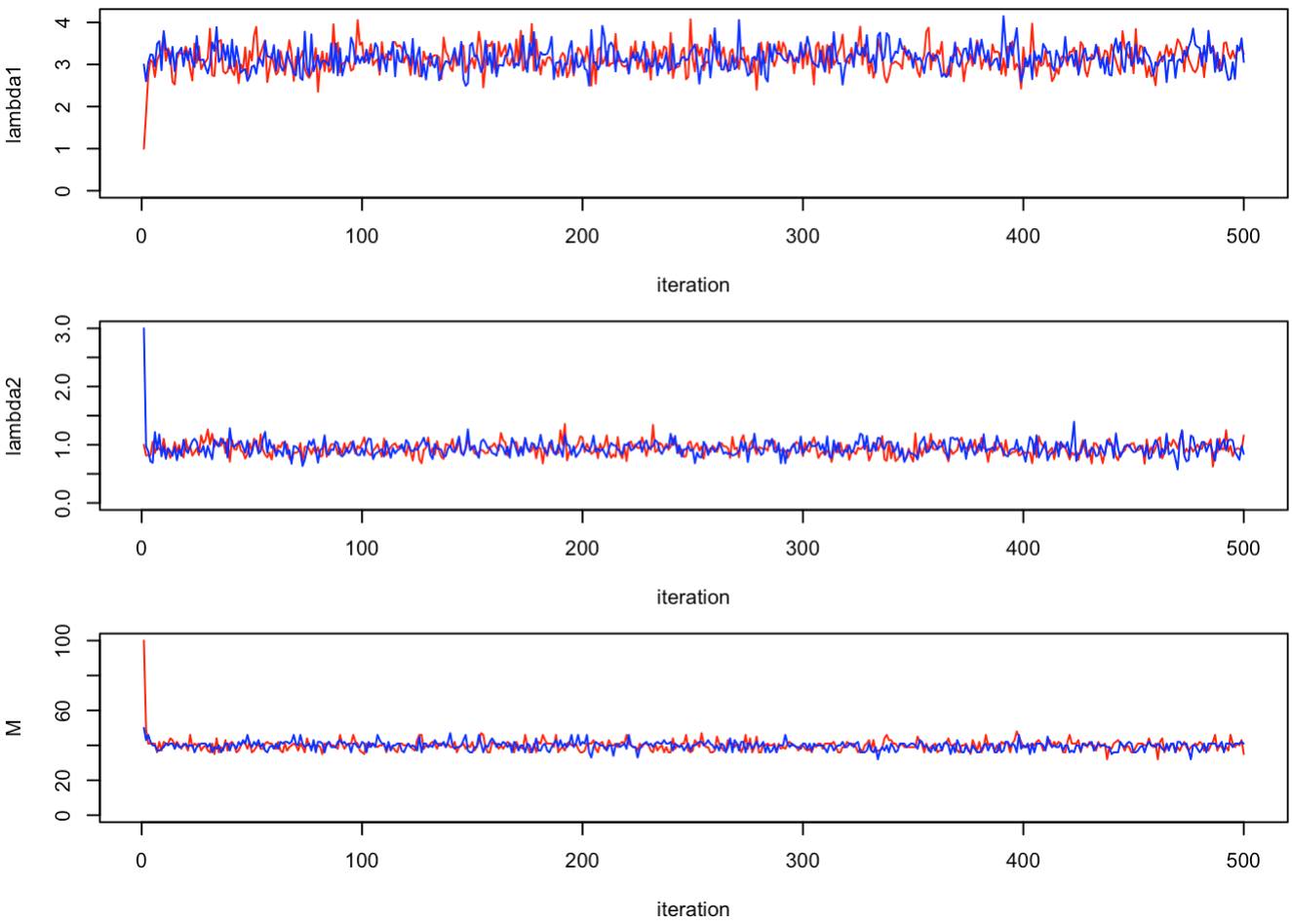
trace plot

The trace plots show the sampled values against the MCMC iteration. The two colours correspond to the two chains started at different initial values.

```
par(mfrow=c(3,1), mar=c(4,4,1,1))
plot(1:nreps, GibbsS1[,1], type="l", col="red", ylim = c(0, max(GibbsS1[,1],GibbsS2[,1])),
     xlab = "iteration", ylab ="lambda1")
points(1:nreps, GibbsS2[,1], type="l", col="blue")

plot(1:nreps, GibbsS1[,2], type="l", col="red", ylim = c(0, max(GibbsS1[,2],GibbsS2[,2])),
     xlab = "iteration", ylab ="lambda2")
points(1:nreps, GibbsS2[,2], type="l", col="blue")

plot(1:nreps, GibbsS1[,3], type="l", col="red", ylim = c(0, max(GibbsS1[,3],GibbsS2[,3])),
     xlab = "iteration", ylab ="M")
points(1:nreps, GibbsS2[,3], type="l", col="blue")
```



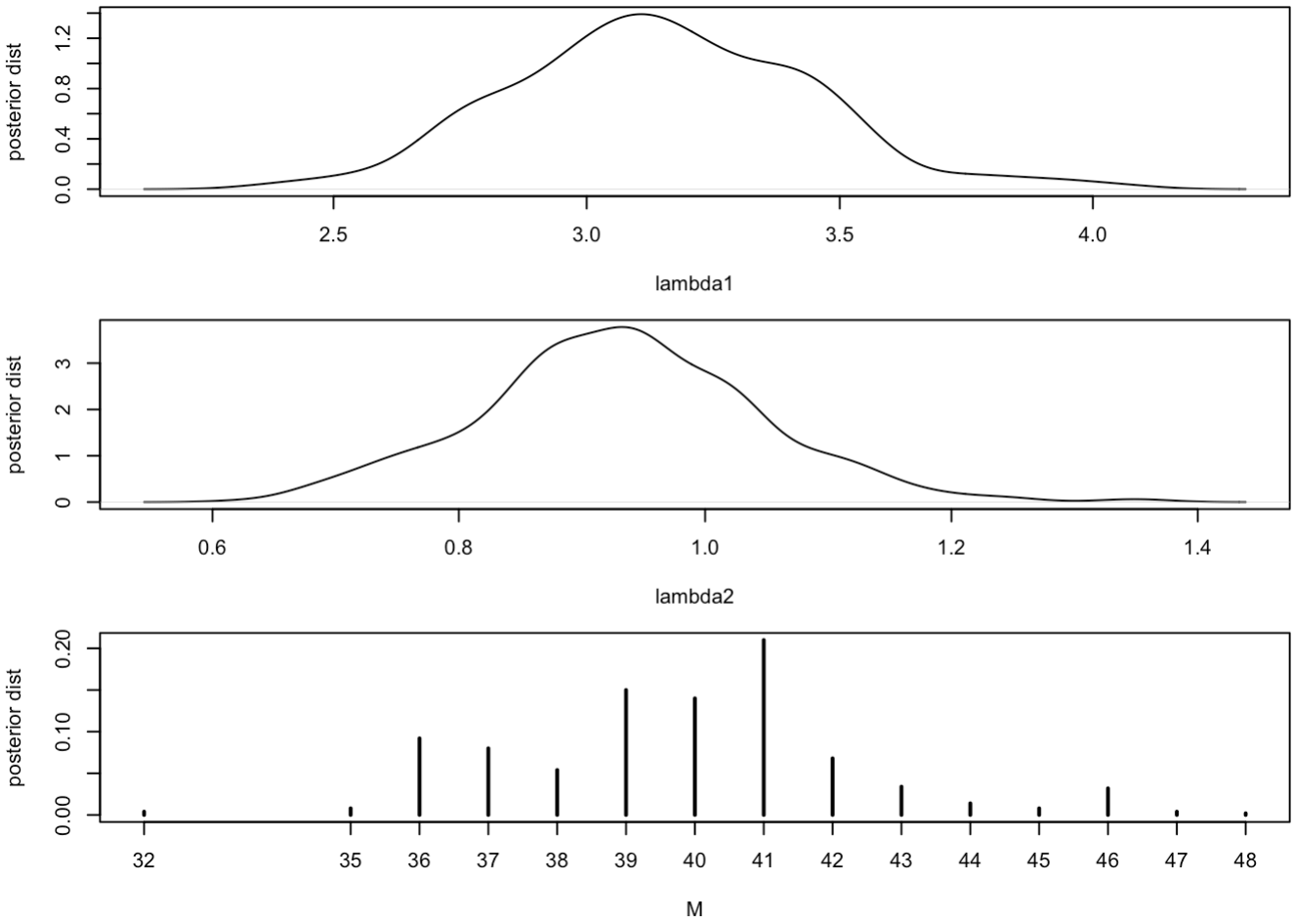
Posterior distribution after discarding the first 50 samples as a burn-in.

After discarding the first 50 iterations, the remaining draws from the chain are used to approximate the marginal posterior distributions.

For the continuous parameters λ_1 and λ_2, the code uses kernel density estimates. For the discrete parameter M, it uses the observed frequencies of the sampled values.

The following code is again exactly the same as in the original Rmd.

```
par(mfrow=c(3,1), mar=c(4,4,1,1))
plot(density(GibbsS1[-(1:50),1]), ylab="posterior dist", xlab="lambda1", main="")
plot(density(GibbsS1[-(1:50),2]), ylab="posterior dist", xlab="lambda2", main="")
plot(table(GibbsS1[-(1:50),3])/nreps, ylab="posterior dist", xlab="M", main="")
```



Autocorrelation

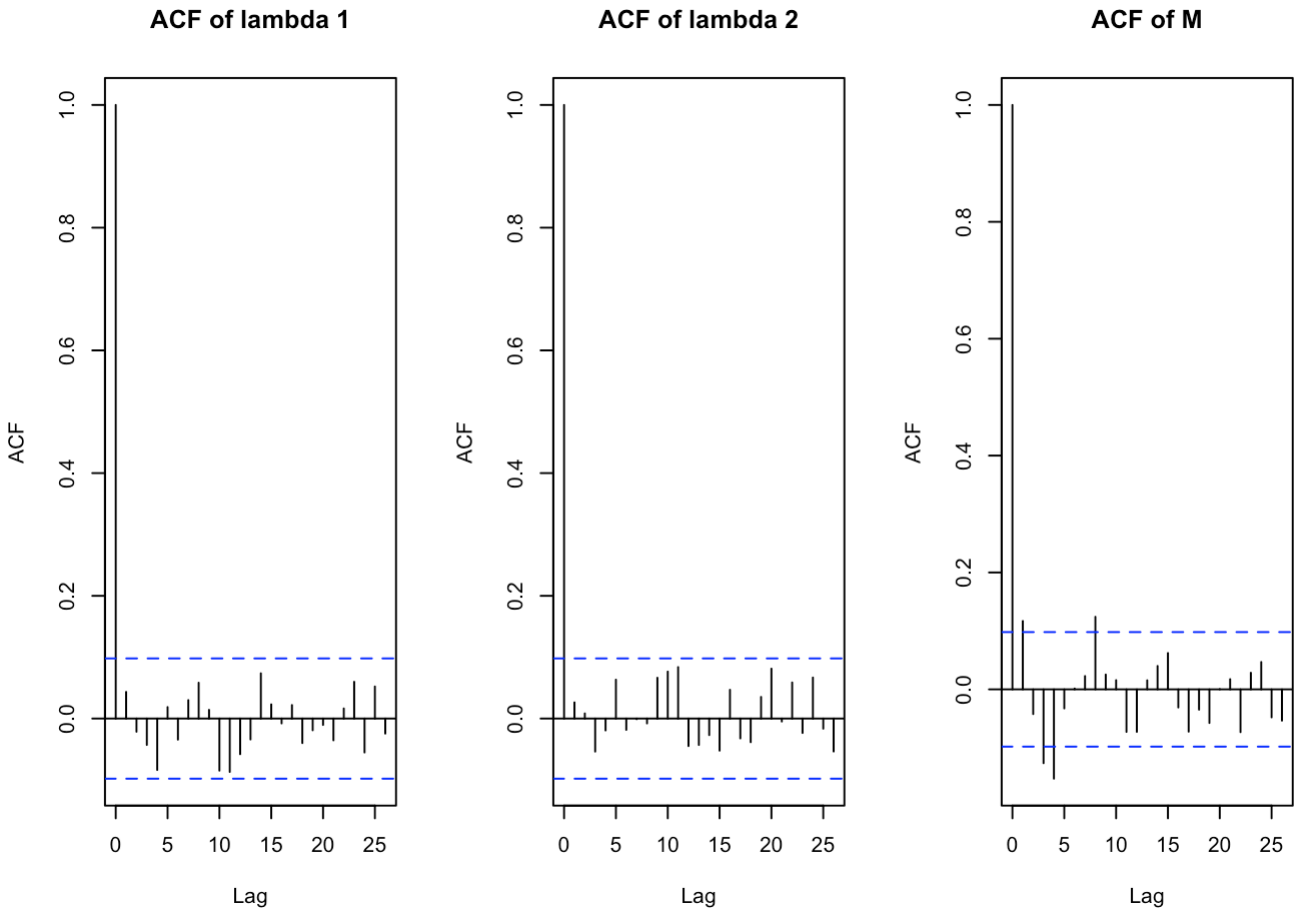
Remove burn-in

```
samples <- GibbsS1[-(1:100), ]
```

Plot autocorrelation

```r
par(mfrow = c(1, 3))

acf(samples[, 1], main = "ACF of lambda 1")
acf(samples[, 2], main = "ACF of lambda 2")
acf(samples[, 3], main = "ACF of M")
```



Thinning example

Remove the first 100 iterations (burn-in)

```r
samples <- GibbsS1[-(1:100), ]

# Keep every 5th sample
thin <- 3 #Search for the bars that are above the CI of the ACF

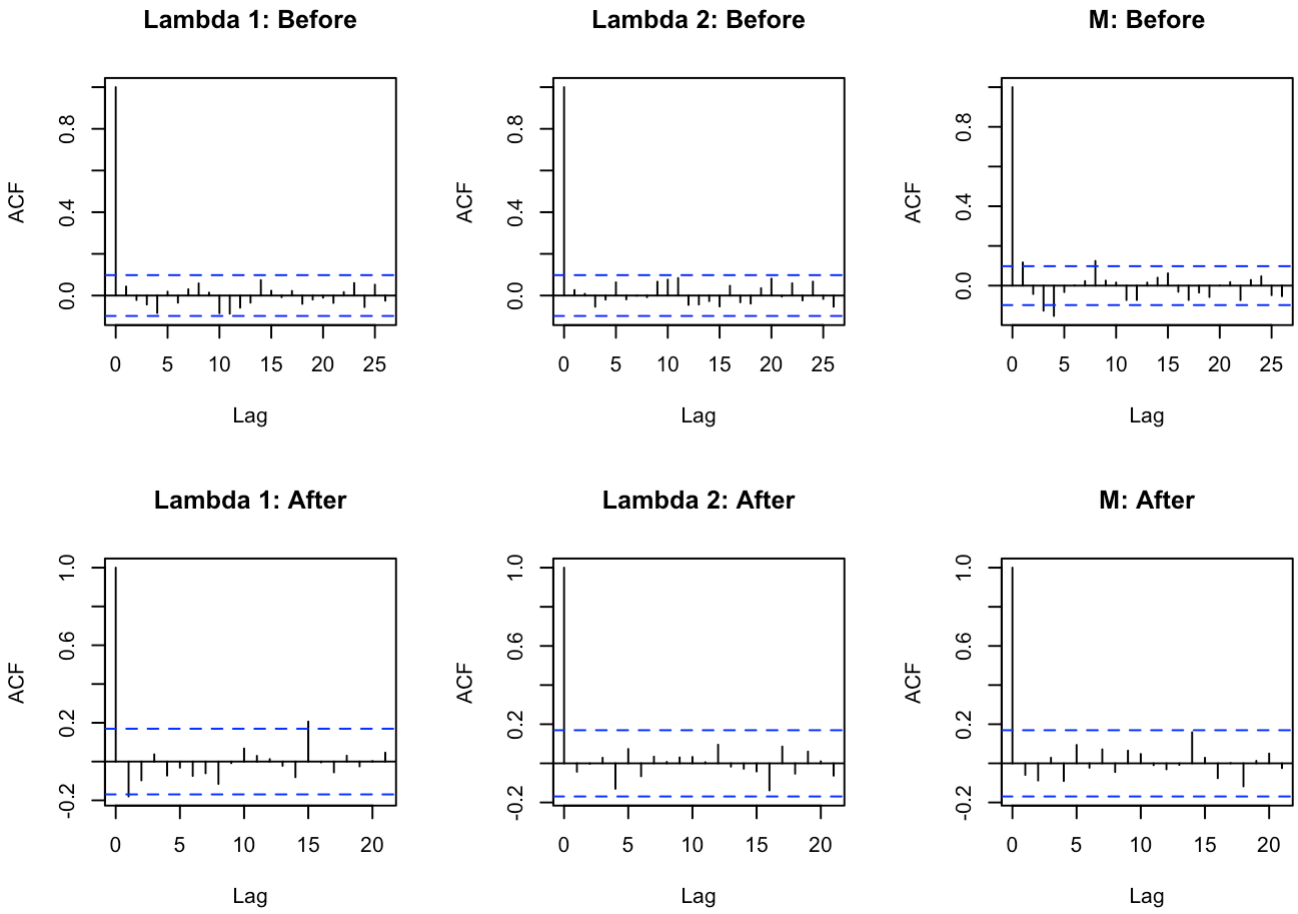
samples_thin <- samples[seq(1, nrow(samples), by = thin), ]

par(mfrow = c(2, 3))

# Before thinning
acf(samples[, 1], main = "Lambda 1: Before")
acf(samples[, 2], main = "Lambda 2: Before")
acf(samples[, 3], main = "M: Before")

# After thinning
acf(samples_thin[, 1], main = "Lambda 1: After")
```

```
acf(samples_thin[, 2], main = "Lambda 2: After")
acf(samples_thin[, 3], main = "M: After")
```



Posterior means before thinning

```
colMeans(samples)
```

```
[1]  3.1485528  0.9324426 39.8350000
```

Posterior means after thinning

```
colMeans(samples_thin)
```

```
[1]  3.1357330  0.9341552 39.8955224
```